

AN EMBEDDED SYSTEM PROJECT REPORT

on

AVR BASIC CLOCK

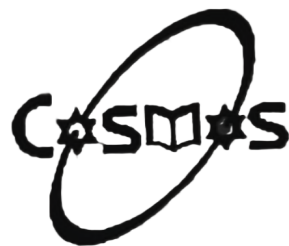
Submitted By

Kushal Prasad Joshi (230345)

Submitted To

Er. Chiranjibi Pandey

in partial fulfilment of requirement for the practicals of
Embedded System (ELX 320) course.



Cosmos College of Management & Technology
(Affiliated to Pokhara University)
Sitapaila, Kathmandu, Nepal

February 1, 2026

ABSTRACT

Time measurement and display systems are fundamental components in many embedded applications. This project presents the design and implementation of a digital clock that displays time in **minutes and seconds (MM:SS)** format using an **AVR ATmega32 microcontroller** and **four multiplexed seven-segment displays**. The system is developed using the **C programming language** and simulated in **Proteus**, with code development carried out in **Microchip Studio**.

The clock utilizes **Timer1 in Clear Timer on Compare (CTC) mode** to generate precise one-second time intervals using the internal RC oscillator of the microcontroller. An interrupt-driven approach is employed to update time variables accurately, while display multiplexing is used to efficiently control multiple seven-segment displays with limited I/O pins. The proposed system demonstrates effective use of timers, interrupts, and port manipulation in AVR-based embedded system design. This project serves as a practical implementation for understanding real-time embedded systems, timer configuration, and display interfacing techniques.

PREFACE

This project report is submitted as a part of the academic requirements for the Embedded System (ELX 320) course. The objective of this project is to apply theoretical knowledge of microcontrollers, timers, interrupts, and digital interfacing to the design of a practical and functional embedded system.

The project focuses on the implementation of a digital clock using the AVR ATmega32 microcontroller and seven-segment displays. Through this work, the author, Kushal Prasad Joshi (230345) gained hands-on experience in AVR architecture, timer configuration, interrupt handling, and display multiplexing techniques. The project was developed and tested using industry-standard tools: Microchip Studio and Proteus, which helped in understanding both software development and hardware-level simulation.

This report has been prepared with the intention of providing clear explanations of the design methodology, hardware and software implementation, and testing procedures. It is hoped that this document will be useful to students and readers who wish to understand the fundamentals of embedded system design using AVR microcontrollers.

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to our respected supervisor, Er. Chiranjibi Pandey, for his continuous support, encouragement, and expert guidance throughout the process of preparing this project report. His valuable feedback and insights have been instrumental in shaping the direction of my work.

I am also thankful to the Department of Information Communication and Technology (ICT) and all the faculty members of Cosmos College of Management & Technology (Affiliated to Pokhara University), Sitapaila, Kathamandu, Nepal for their continuous support and for providing me with the opportunity and resources to carry out this proposed project.

I would also like to express my kind regards to the people around me who have directly or indirectly contributed to the successful completion of this report. Also I will thank my college friends who gave me valuable suggestions and feedback during the preparation of this project report.

Parts of this report were drafted and refined with the assistance of AI-powered language models, including ChatGPT [1]. The AI tools were used solely to help with structuring, phrasing, and clarity of the text. All research, analysis, design, and conclusions presented in this report are entirely the author's own work.

Finally, I extend my sincere thanks to my family and friends for their unwavering support and encouragement during this endeavour.

I am grateful to all of you.

Kushal Prasad Joshi (230345)

TABLE OF CONTENTS

Abstract	ii
Preface	iii
Acknowledgement	iv
Table of Content	vi
List of Figures	vii
List Of Tables	viii
List of Abbreviations	ix
1 INTRODUCTION	1
1.1 BACKGROUND AND MOTIVATION	1
1.2 PROBLEM STATEMENT	1
1.3 PROJECT OBJECTIVES	1
1.4 SCOPE OF THE PROJECT	2
1.5 SIGNIFICANCE OF THE PROJECT	2
2 LITERATURE REVIEW	3
3 METHODOLOGY	4
3.1 OVERVIEW OF PROPOSED METHODOLOGY	4

3.2	WATERFALL DEVELOPMENT MODEL	4
3.3	ARCHITECTURE OF PROPOSED SYSTEM	5
4	TIME SCHEDULE	6
4.1	PROJECT TIMELINE	6
4.2	DELIVERABLES	6
5	CONCLUSION AND FUTURE WORK	7
5.1	CONCLUSION	7
5.2	FUTURE WORK	7
A	RISK ANALYSIS AND MITIGATION	
A.1	RISK, IMPACT AND MITIGATION STRATEGY	
A.2	SUMMARY	
B	TECHNICAL DETAILS	
B.1	PROTEUS SCHEMATIC	
B.2	CLOCK PROGRAM	
	GLOSSARY	
	REFERENCES	

LIST OF FIGURES

3.3	System Architecture of Digital Clock System	5
4.1	Gantt Chart of Project Activities	6
B.1	Proteus Schematic of Digital Clock System	

LIST OF TABLES

A.1	Risk Analysis and Mitigation Strategies
-----	---

LIST OF ABBREVIATIONS

AVR	Advanced Virtual RISC
CTC	Clear Timer on Compare
I/O	Input Output
MM:SS	Minutes:Seconds
RT	Resistor Capacitor
RTC	Real-Time Clock

CHAPTER 1: INTRODUCTION

Embedded systems play a crucial role in modern electronic devices by integrating hardware and software to perform dedicated functions efficiently. Microcontroller-based systems are widely used in applications such as digital clocks, timers, control systems, and monitoring devices due to their low cost, reliability, and real-time processing capabilities. This project focuses on the design and implementation of a digital clock using an AVR microcontroller and seven-segment displays to demonstrate fundamental embedded system concepts.

1.1 BACKGROUND AND MOTIVATION

Digital clocks are one of the most common real-time embedded applications and serve as an excellent platform for learning microcontroller programming. The motivation behind this project is to gain practical exposure to embedded system design by interfacing a microcontroller with external display hardware and implementing precise time measurement using internal timers.

The AVR ATmega32 microcontroller provides built-in hardware timers, interrupt handling mechanisms, and configurable I/O ports, making it suitable for implementing timing-based applications. By developing a digital clock that displays time in minutes and seconds format, this project aims to strengthen understanding of timer configuration, interrupt-driven programming, and display multiplexing techniques.

1.2 PROBLEM STATEMENT

The objective of this project is to design and implement a digital clock capable of accurately displaying time in **MM:SS format** using an AVR ATmega32 microcontroller. The system must generate precise one-second intervals, update time values reliably, and display the output using four seven-segment displays while efficiently utilizing the available microcontroller I/O pins.

1.3 PROJECT OBJECTIVES

The main objectives of the project are:

- To design a digital clock using the AVR ATmega32 microcontroller.

- To generate accurate one-second timing using hardware timers.
- To implement interrupt-driven time updating for reliable operation.
- To interface and control multiple seven-segment displays using multiplexing.
- To simulate and verify the system using Proteus software.
- To develop and test the embedded C program using Microchip Studio.

1.4 SCOPE OF THE PROJECT

The scope of this project is limited to the design and simulation of a digital clock displaying minutes and seconds using four seven-segment displays. The system uses the internal RC oscillator of the microcontroller for timing purposes and does not include external time calibration mechanisms.

The project does not cover advanced features such as hour display, alarm functionality, real-time clock (RTC) modules, or power optimization techniques. However, the design can be extended in the future to incorporate additional features and hardware components.

1.5 SIGNIFICANCE OF THE PROJECT

This project is significant as it provides a practical understanding of real-time embedded system design using AVR microcontrollers. It demonstrates the effective use of timers, interrupts, and I/O port manipulation in a real-world application.

Additionally, the project serves as a foundational learning exercise for students studying embedded systems, as the concepts applied in this work are directly transferable to more complex applications such as industrial timers, digital control systems, and consumer electronic devices.

CHAPTER 2: LITERATURE REVIEW

Several studies and projects have explored the design and implementation of digital clocks and time-based embedded systems using microcontrollers. These works primarily focus on accurate time generation, efficient display control, and reliable system operation.

Microcontroller-based digital clocks are commonly implemented using hardware timers and interrupt mechanisms. According to Mazidi et al. [2], AVR microcontrollers provide flexible timer modules that can be configured in various modes, including Clear Timer on Compare (CTC) mode, which is suitable for precise time interval generation. The use of interrupts ensures that timing accuracy is maintained independently of the main program execution.

Seven-segment displays are widely used in embedded applications due to their simplicity, low cost, and ease of interfacing. Barret and Pack [3] describe multiplexing as an efficient technique for controlling multiple seven-segment displays using a limited number of I/O pins. By rapidly switching between displays, multiplexing creates the illusion of continuous illumination while reducing hardware complexity.

Several embedded system designs utilize internal oscillators to reduce external component count and simplify circuit design. As discussed by the Atmel datasheet [4], the internal RC oscillator of the ATmega32 microcontroller is suitable for applications where ultra-high timing precision is not critical, such as basic digital clocks and timers.

Simulation tools such as Proteus have been extensively used for validating embedded system designs before hardware implementation. Proteus allows real-time simulation of microcontroller-based circuits, enabling developers to test both firmware and hardware interaction [5]. Similarly, Microchip Studio provides a robust development environment for AVR microcontrollers, supporting embedded C programming, debugging, and code optimization [6].

Based on the reviewed literature, it is evident that implementing a digital clock using AVR microcontrollers, hardware timers, interrupts, and multiplexed seven-segment displays is a well-established and reliable approach. This project follows proven design methodologies while serving as a practical learning platform for embedded system development.

CHAPTER 3: METHODOLOGY

This chapter describes the methodology adopted for the design and implementation of the digital clock system. It explains the overall development approach, the system architecture, and the sequence of steps followed to achieve the project objectives.

3.1 OVERVIEW OF PROPOSED METHODOLOGY

The proposed methodology follows a systematic approach to develop a microcontroller-based digital clock using the AVR ATmega32. The design process begins with requirement analysis, followed by hardware and software design, implementation, and testing.

The system is developed using an interrupt-driven approach to ensure accurate time measurement. Hardware timers are configured to generate precise one-second intervals, and multiplexing is employed to control multiple seven-segment displays efficiently. Simulation using Proteus is performed to validate the design before final testing.

3.2 WATERFALL DEVELOPMENT MODEL

The Waterfall development model is adopted for this project due to its structured and sequential nature, which is suitable for small-scale embedded system applications with well-defined requirements.

The stages of the Waterfall model used in this project are as follows:

- **Requirement Analysis:** Identification of functional and non-functional requirements of the digital clock system.
- **System Design:** Design of hardware connections and software logic, including timer configuration and display multiplexing.
- **Implementation:** Development of embedded C code using Microchip Studio and creation of the circuit in Proteus.
- **Testing:** Verification of correct time counting and display operation through simulation.
- **Maintenance:** Provision for future modifications and enhancements.

The Waterfall model ensures clear documentation at each stage and simplifies project management, making it appropriate for academic projects.

3.3 ARCHITECTURE OF PROPOSED SYSTEM

The architecture of the proposed digital clock system consists of three main components: the microcontroller unit, the display unit, and the power supply.

The AVR ATmega32 microcontroller serves as the core processing unit of the system. It is responsible for generating time intervals using Timer1 configured in Clear Timer on Compare (CTC) mode and handling interrupts to update time variables. The microcontroller also controls the multiplexing of four seven-segment displays through its I/O ports.

The display unit comprises four common-cathode seven-segment displays arranged to represent minutes and seconds in MM:SS format. Segment lines are connected to a common port through current-limiting resistors, while individual digit enable lines are used to activate each display sequentially.

The power supply provides a regulated 5V DC voltage required for the operation of the microcontroller and display unit. Figure 3.3 illustrates the overall architecture of the proposed system.

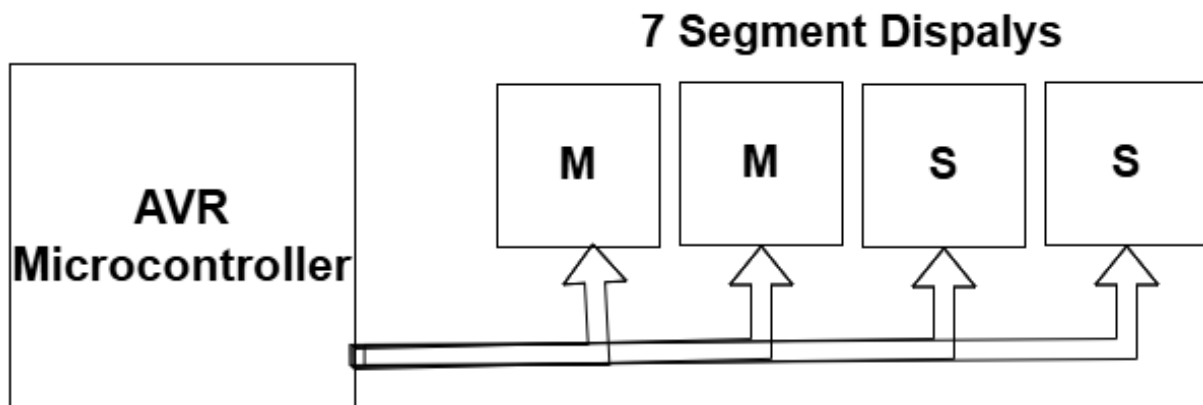


Figure 3.3 : System Architecture of Digital Clock System

CHAPTER 4: TIME SCHEDULE

Time analysis is performed to estimate and manage the duration required for each phase of the project. Proper scheduling ensures timely completion of the project and effective utilization of available resources.

4.1 PROJECT TIMELINE

The development of the digital clock project was divided into multiple phases based on the Waterfall development model. Each phase was allocated a specific time period, as summarized in Gantt Chart 4.1 .

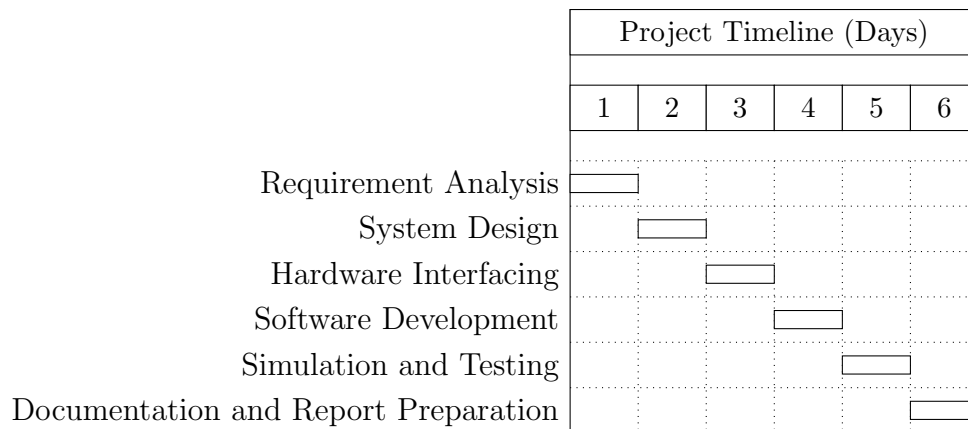


Figure 4.1 : Gantt Chart of Project Activities

The time allocation was sufficient to complete the project within the academic schedule. Minor adjustments were made during the testing phase to resolve timing and display-related issues.

4.2 DELIVERABLES

The major deliverables produced during the course of the project include:

- Source codes.
- Working prototype of the system (Simulation).
- Final project report.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This project successfully demonstrated the design and implementation of a digital clock displaying time in minutes and seconds (MM:SS) using the AVR ATmega32 microcontroller. The system was developed using embedded C programming and verified through simulation in Proteus, with code development carried out in Microchip Studio.

Accurate time measurement was achieved by configuring Timer1 in Clear Timer on Compare (CTC) mode and utilizing interrupt-driven programming. The use of multiplexing enabled efficient control of four seven-segment displays while minimizing the number of required I/O pins. The system operated reliably and met all the defined functional and non-functional requirements.

Through this project, practical knowledge of AVR microcontroller architecture, timer configuration, interrupt handling, and display interfacing was gained. The project served as an effective learning platform for understanding real-time embedded system concepts and their practical applications.

5.2 FUTURE WORK

Although the current system fulfills its intended objectives, several enhancements can be incorporated in future developments to improve functionality and performance. Possible future extensions of the project include:

- Extending the display format to include hours (HH:MM:SS).
- Integrating a Real-Time Clock (RTC) module for improved time accuracy.
- Adding user input buttons for time setting and reset functionality.
- Implementing alarm or timer features.
- Deploying the system on physical hardware for real-world testing.
- Optimizing power consumption for battery-operated applications.

These enhancements would make the system more versatile and suitable for a wider range of real-time embedded applications.

CHAPTER A: RISK ANALYSIS AND MITIGATION

Risk analysis is the process of identifying potential issues that may affect the successful completion of a project and defining strategies to mitigate their impact. In embedded system projects, risks may arise from both hardware and software components.

Table [A.1](#) summarizes the identified risks associated with the **AVR Basic Clock** project along with their mitigation strategies.

A.1 RISK, IMPACT AND MITIGATION STRATEGY

Risk	Impact	Mitigation Strategy
Incorrect timer configuration	Inaccurate time display	Careful calculation of timer values and verification using simulation
Display flickering due to improper multiplexing	Poor visual output	Adjusting refresh delay and ensuring correct digit enable logic
Internal oscillator inaccuracy	Time drift over long duration	Acceptable tolerance for academic project and proper prescaler selection
Hardware wiring errors	System malfunction	Verification of circuit connections using Proteus before testing
Software bugs or logic errors	Unexpected system behavior	Incremental testing and use of interrupt-driven design

Table A.1 : Risk Analysis and Mitigation Strategies

A.2 SUMMARY

The identified risks were effectively managed through careful design, simulation-based testing, and incremental debugging. As a result, the project was completed successfully with reliable performance.

CHAPTER B: TECHNICAL DETAILS

B.1 PROTEUS SCHEMATIC

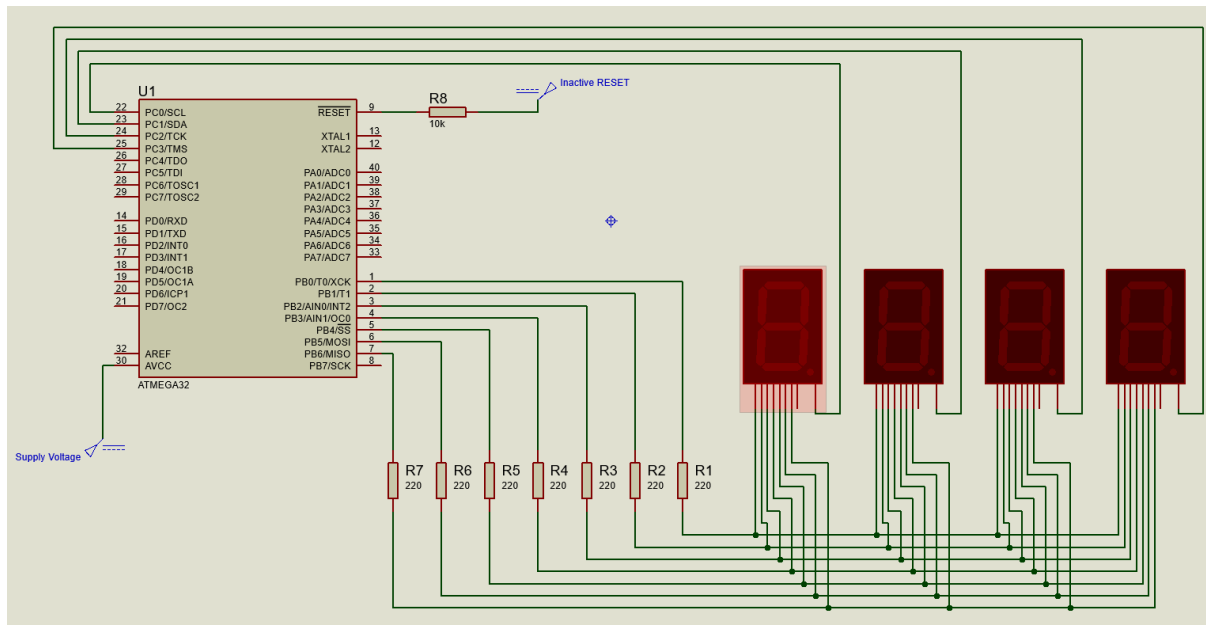


Figure B.1 : Proteus Schematic of Digital Clock System

B.2 CLOCK PROGRAM

```
1 /*
2  * clock-program.c
3  *
4  * Created: 1/29/2026 6:46:44 PM
5  * Author : Kushal Prasad Joshi
6  *
7  * Description:
8  * This program implements a digital clock (MM:SS) using
9  * ATmega32.
10 * A 4-digit multiplexed 7-segment display is driven using
11 * PORTB and PORTC.
12 * Timer1 is configured in CTC mode to generate a 1-second
13 * interrupt.
14 */
```

```

12
13 #define F_CPU 1000000UL    // CPU frequency = 1 MHz
14 // Using internal RC oscillator with CKDIV8 enabled
15 // This definition is REQUIRED for correct delay calculations
16
17 #include <avr/io.h>        // AVR register definitions
18 #include <avr/interrupt.h> // Interrupt handling
19 #include <util/delay.h>    // Delay functions (_delay_ms)
20
21 /*
22  * volatile is VERY IMPORTANT here.
23  *
24  * Why volatile?
25  * -----
26  * 'seconds' and 'minutes' are modified inside an ISR (
27  *   Interrupt Service Routine)
28  * and also read inside the main program.
29  *
30  * Without 'volatile', the compiler may assume these
31  *   variables never change
32  * unexpectedly and may optimize them incorrectly.
33  *
34  * volatile tells the compiler:
35  * "These variables can change at any time - always read them
36  *   from memory."
37  */
38
39 volatile uint8_t seconds = 0;
40 volatile uint8_t minutes = 0;
41
42 /*
43  * Lookup table for 7-segment display
44  *
45  * Each value represents segments a-g for digits 0-9
46  * Bit pattern corresponds to:
47  * bit0 = a, bit1 = b, ..., bit6 = g
48  *
49  * Example:
50  * 0x3F -> 0b00111111 -> displays digit '0'
51  */

```

```

48 uint8_t seg_code[10] = {
49     0x3F,  // 0
50     0x06,  // 1
51     0x5B,  // 2
52     0x4F,  // 3
53     0x66,  // 4
54     0x6D,  // 5
55     0x7D,  // 6
56     0x07,  // 7
57     0x7F,  // 8
58     0x6F   // 9
59 };
60
61 /*
62  * TIMER1 Compare Match Interrupt Service Routine
63  *
64  * This ISR is executed every 1 second.
65  * It increments seconds and minutes like a real clock.
66  */
67 ISR(TIMER1_COMPA_vect)
68 {
69     seconds++;           // Increment seconds every
70                          // interrupt
71
72     if (seconds == 60) { // If 60 seconds reached
73         seconds = 0;     // Reset seconds
74         minutes++;       // Increment minutes
75
76         if (minutes == 60) // If 60 minutes reached
77             minutes = 0;   // Reset minutes
78     }
79 }
80
81 /*
82  * Timer1 initialization function
83  *
84  * Configures Timer1 to generate an interrupt every 1 second
85  */
86 void timer1_init(void)

```

```

86 {
87     TCCR1B |= (1 << WGM12);      // Configure Timer1 in CTC
        mode
88     // CTC = Clear Timer on Compare Match
89
90     OCR1A = 976;                  // Compare value for 1 second
91     /*
92      * Calculation:
93      * Timer frequency = 1 MHz / 1024 = 976.56 Hz
94      * 1 second ? 976 counts
95      */
96
97     TCCR1B |= (1 << CS12) | (1 << CS10);
98     // Prescaler = 1024
99
100    TIMSK |= (1 << OCIE1A);      // Enable Timer1 Compare Match
        A interrupt
101 }
102
103 /*
104  * Function to display a single digit at a given position
105  *
106  * digit : number to display (0-9)
107  * pos   : digit position (0 to 3)
108  *
109  * Multiplexing technique is used:
110  * - One digit is enabled at a time
111  * - Digits are switched fast enough to appear continuous
112  */
113 void display_digit(uint8_t digit, uint8_t pos)
114 {
115     // Activate only the required display (PC0-PC3 are active
        LOW)
116     PORTC = (PORTC & 0xF0) | (~(1 << pos) & 0x0F);
117
118     // Output segment pattern to PORTB
119     PORTB = seg_code[digit];
120
121     // Small delay for persistence of vision

```

```

122     _delay_ms(5);
123 }
124
125 int main(void)
126 {
127     /*
128      * PORTB:
129      * PB0-PB6 -> segments a-g (output)
130      */
131     DDRB = 0x7F;    // 0b01111111 ? PB0-PB6 as output
132
133     /*
134      * PORTC:
135      * PC0-PC3 -> digit enable lines (output)
136      */
137     DDRC = 0x0F;    // 0b00001111 ? PC0-PC3 as output
138
139     // Initial states
140     PORTB = 0x00;    // Turn OFF all segments
141     PORTC = 0x0F;    // Disable all displays (active LOW)
142
143     timer1_init(); // Initialize Timer1
144     sei();         // Enable global interrupts
145
146     while (1)
147     {
148         /*
149          * Display format:
150          * [MM][SS]
151          *
152          * Digit positions:
153          * pos 0 ? minutes tens
154          * pos 1 ? minutes units
155          * pos 2 ? seconds tens
156          * pos 3 ? seconds units
157          */
158
159         display_digit(minutes / 10, 0);
160         display_digit(minutes % 10, 1);

```

```
161         display_digit(seconds / 10, 2);
162         display_digit(seconds % 10, 3);
163     }
164 }
```

GLOSSARY

Microcontroller: A compact integrated circuit designed to govern a specific operation in an embedded system.

Embedded System: A combination of hardware and software designed to perform a dedicated function within a larger system.

Seven-Segment Display: An electronic display device used to display decimal numerals using seven individual LED segments.

Multiplexing: A technique that allows multiple display units to share common data lines by activating them sequentially.

Hardware Timer: A built-in peripheral in a microcontroller used to measure time intervals and generate delays.

Interrupt: A signal that temporarily halts the main program execution to execute a specific service routine.

CTC Mode: Clear Timer on Compare mode used to reset the timer counter when a predefined value is reached.

Internal RC Oscillator: An on-chip clock source that generates timing signals without the need for external components.

Proteus: A simulation software used for designing and testing electronic circuits and embedded systems.

Microchip Studio: An integrated development environment for developing and debugging AVR microcontroller applications.

REFERENCES

- [1] OpenAI, “Chatgpt (gpt-5.1),” aI assistant used for drafting and analysis. [Online]. Available: <https://chat.openai.com>
- [2] M. A. Mazidi, S. Naimi, and S. Naimi, *AVR Microcontroller and Embedded Systems: Using Assembly and C*. Pearson Education, 2014.
- [3] S. F. Barrett and D. J. Pack, *Atmel AVR Microcontroller Primer: Programming and Interfacing*. Morgan & Claypool Publishers, 2012.
- [4] *ATmega32 Datasheet*, Atmel Corporation, 2016.
- [5] “Proteus design suite,” Labcenter Electronics, simulation software for electronic circuits.
- [6] “Microchip studio for avr and sam devices,” Microchip Technology Inc., integrated Development Environment.